UNIVERSITY OF ENGINEERING & TECHNOLOGY, MARDAN.

OPEN ENDED LAB REPORT

EvoPet: Interactive Virtual Pet Game

CS-103L Object Oriented Programming Lab

Department	Telecommunication Engineering
Subject	CS-103L Object Oriented Programming Lab
Semester	2nd – Spring 2026
Assigned On	29/04/2026
Submission Due	15/05/2026
Target CLO/PLO	CLO-1,2,3,4 | PLO-1,4,5,8
Taxonomy Level	C-4 P-3,4 A-3

Group Members

Student Name	Reg. Number
Muhammad Shaheer Saif	25MDTLE458
Muhammad Mustafa	25MDTLE454
Muhammad Tehzeeb Ul Hassan	25MDTLE482

1. Introduction / Related Theory

Object-Oriented Programming (OOP) is a widely-adopted programming paradigm that structures software around objects and classes, enabling organized, reusable, and modular code design. OOP promotes concepts such as encapsulation, inheritance, polymorphism, and abstraction, which collectively simplify complex software systems.

In this project, we developed EvoPet — an Interactive Virtual Pet Game — where a user can adopt and take care of a digital pet. The pet maintains dynamic attributes such as hunger, happiness, and energy, all of which change in real time based on how the user interacts with it.

Unlike traditional virtual pet simulations that rely on static, fixed responses, EvoPet introduces a behavior-based personality system. The pet's personality evolves dynamically depending on the frequency and type of actions the user performs over the course of gameplay. This adaptive mechanism provides a more engaging and realistic user experience.

1.1 Key OOP Concepts Applied

- Encapsulation: All pet attributes and behaviors are bundled within a single Pet class.
- Abstraction: The user interacts through a simple menu without needing to understand internal logic.
- Data Hiding: Private member variables are accessed and modified only through class methods.
- Object Instantiation: A Pet object is created at runtime based on user input.

2. Objectives

- To implement core OOP concepts including classes, objects, and encapsulation in a practical application.
- To simulate a virtual pet with dynamic, real-time behavioral changes.
- To create an interactive system where each user action directly affects the pet's internal state.
- To design and implement a unique adaptive personality system where the pet's character evolves based on past user interactions.
- To demonstrate the practical application of C++ programming through a structured and well-documented project.

3. Concept Explanation (Core Logic)

The core innovation of EvoPet lies in its adaptive behavior system. The pet does not simply react to isolated actions — it learns from and responds to patterns in user behavior over time.

3.1 Behavior Tracking

Each user action is tracked by an internal counter inside the Pet class:

- feedCount — incremented each time the user feeds the pet.
- playCount — incremented each time the user plays with the pet.
- trainCount — incremented each time the user trains the pet.
- neglectCount — incremented each time the user ignores the pet.

3.2 Personality Evolution

When a specific counter crosses a defined threshold, the pet's personality is updated via the updatePersonality() method. The following table illustrates this mapping:

User Behavior	Resulting Personality	Effect on Pet
Frequent Feeding	**Lazy**	Pet becomes less active; reduced energy consumption
Frequent Playing	**Friendly**	Pet is cheerful and responds warmly to all interactions
Frequent Training	**Intelligent**	Pet learns faster and displays smart responses
Frequent Neglect	**Aggressive**	Pet becomes hostile and unresponsive

This creates a simple but effective adaptive system, ensuring that different users who interact differently with the pet will obtain distinct outcomes — making every gameplay session unique.

4. System Design

4.1 Class: Pet

The entire logic of the game is encapsulated within a single class named Pet. Below is a description of its attributes:

Attribute	Description
`name`	Stores the pet's name (string)
`hunger`	Integer (0–100); increases over time without feeding
`happiness`	Integer (0–100); affected by play and neglect
`energy`	Integer (0–100); restored by sleep, reduced by play
`personality`	String; evolves based on user actions
`feedCount`	Counter tracking number of feed actions
`playCount`	Counter tracking number of play actions
`trainCount`	Counter tracking number of train actions
`neglectCount`	Counter tracking number of ignore actions

4.2 Member Functions

- feed() — Decreases the hunger attribute and slightly increases happiness.
- play() — Increases happiness but reduces energy level.
- sleep() — Restores the pet's energy to a higher level.
- train() — Simulates skill development; increments trainCount.
- neglect() — Reduces happiness and increments neglectCount.
- updatePersonality() — Evaluates all counters and assigns an appropriate personality.
- showStatus() — Displays the current values of all pet attributes to the user.

5. Step-by-Step Procedure

The program follows a simple, iterative input-output loop as described below:

1. The program initializes and prompts the user to enter the pet's name.
2. A Pet object is instantiated with default attribute values (hunger = 50, happiness = 50, energy = 100, personality = Neutral).
3. A menu is displayed offering the following interaction options:

- (1) Feed the pet
- (2) Play with the pet
- (3) Let the pet sleep
- (4) Train the pet
- (5) Ignore the pet
- (6) Show current status
- (0) Exit the game

1. The user selects an action. The corresponding method is called on the Pet object.
2. Pet attributes are updated and the relevant action counter is incremented.
3. updatePersonality() is called after each action to re-evaluate the pet's personality.
4. The loop continues, showing the menu again, until the user chooses to exit (option 0).

6. Unique Features of the Project

- **Adaptive Personality System: The pet's personality changes dynamically based on cumulative user behavior, not isolated events.**
- **Multi-Attribute Management: The system simultaneously tracks and balances three independent attributes (hunger, happiness, energy), adding gameplay depth.**
- **Personality-Based Responses: The pet reacts differently to the same action based on its current personality, making interactions feel realistic.**
- **Minimalist Yet Effective Design: The system achieves meaningful adaptivity using simple counter-threshold logic, demonstrating efficient OOP design.**
- **Extensibility: The architecture is modular, making it straightforward to add GUI elements, multiple pets, or advanced AI behaviors in future iterations.**

These features collectively set EvoPet apart from conventional virtual pet programs that operate on fixed, predictable rule sets.

7. Results and Discussion

The program was implemented and tested in Dev C++ (C++14 standard). During testing, the system successfully demonstrated all intended behaviors and state transitions.

7.1 Observed Outcomes

- Repeated feeding actions correctly transitioned the pet's personality to 'Lazy' after five consecutive feeds.
- Consistent play interactions resulted in the 'Friendly' personality, with noticeably positive response messages.
- Training the pet five or more times triggered the 'Intelligent' personality, demonstrating counter-threshold logic.
- Ignoring the pet repeatedly led to the 'Aggressive' state, reducing both happiness and responsiveness.
- The showStatus() function accurately reflected all attribute changes in real time.

7.2 Discussion

The results confirm that counter-based adaptive logic is a practical and computationally inexpensive way to simulate behavioral evolution in a game entity. The personality system makes each session unique — two users who interact differently with the same pet will experience fundamentally different outcomes.

One limitation identified during testing is that the system currently updates only a single dominant personality. Future improvements could include weighted combinations of multiple personality traits, providing even richer behavioral diversity.

8. Conclusion

This project demonstrates the effective and practical use of Object-Oriented Programming to build an interactive, adaptive simulation. Through the design and implementation of the Pet class — complete with encapsulated attributes, behavioral methods, and a counter-driven personality engine — we successfully realized the full scope of the OEL requirements.

The behavior-based personality system adds a layer of uniqueness that distinguishes EvoPet from standard virtual pet implementations. The project reinforces core OOP

principles and showcases how thoughtful design choices — even with simple logic — can produce engaging, realistic systems.

Future enhancements may include a graphical user interface (GUI), support for multiple pets with inter-pet interactions, a persistence layer to save pet state between sessions, and more sophisticated AI-driven behavioral models.

Key Takeaway

"This project simulates a virtual pet whose personality evolves based on repeated user interactions, demonstrating adaptive behavior using OOP concepts."

Declaration

We hereby declare that this report is our own original work. All external sources have been cited appropriately. This report has not been submitted previously for any other course or assessment.